VIDEO THREAT DETECTION PLATFORM

SentinelAI

It watches the video, finds the incident,
and shows the proof.

STUDENT	SUPERVISOR	DEPARTMENT	SESSION
[your name + roll no]	[supervisor name]	[dept / university]	[2025 – 2026]

6 DETECTION MODELS · 5 INCIDENT TYPES · LIVE PROGRESS · CHAT OVER RESULTS · CCTV-READY DESIGN

One video in. A clear incident report out.

You give SentinelAI a video. It gives back a report: **what happened, at what second, how sure it is**, and a short clip with boxes drawn on it as proof.

Inside, it does not use one model. It uses **six** — two that find objects, two that read actions over time, and two vision-language models that understand the scene. The user never picks a model. The system picks, and stores which model actually gave the answer.

Around this sits a full application: user accounts, video storage, a background job queue so the browser never freezes, a clean database, and an admin panel.

> **CCTV.** Today the system analyses uploaded video files. The pipeline was built so a **live CCTV (RTSP) camera stream** can be plugged in as just another video source — the worker only needs frames. So connecting real cameras is the next step, not a rebuild.

TECHNOLOGY USED

Frontend	Next.js 15 · React
API	FastAPI · JWT login
Worker	ARQ background queue
Database	PostgreSQL · Prisma
Cache / events	Redis 7
File storage	MinIO (S3)
Vision models	YOLOv8n · OWLv2 · VideoMAE ×2
Language-vision	Gemini · Qwen2.5-VL
Deployment	Docker Compose

IMAGE 00

Main screenshot — results page with an incident found

/results

The product shot for the opening minute of the presentation.

Five incident types, one report format

Every model, however different inside, returns the **same output shape** — a yes/no flag, a plain summary, and a list of incidents with start, peak and end times. That is what makes six different models swappable.

TYPE	WHAT COUNTS	WHICH MODELS CAN SEE IT
violence	Fighting, assault, shooting, explosion, visible weapons	All six
theft	Snatching, mugging, burglary, shoplifting	Action model, Gemini, Qwen-VL
harassment	Unwanted contact, following, intimidation	**Only the two language-vision models**
accident	Road crashes, a person hit or falling	Action model, Gemini, Qwen-VL
other	Fire, arson, vandalism	Action model, Gemini, Qwen-VL

Harassment is why the design looks like this. It is not an object and not a pose — it is intent and persistence, read from context. A normal classifier with a fixed label list simply cannot produce it. That is why a language-vision model sits at the top of the system.

DETECTION MODELS	INCIDENT TYPES	LIVE EVENT TYPES	DATABASE TABLES
6	**5**	**7**	**18+**
2 object · 2 action · 2 VLM	Shared output format	Streamed to the browser	Normalised to 3NF

Cameras record everything. Nobody watches anything.

- **CCTV is recorded, not watched.** Footage is usually opened only *after* something is reported. A human cannot stay alert across many screens, and real incidents are rare inside hours of normal video.[1] [6]
- **Models answer a smaller question than people ask.** A weapon detector says "gun in frame". An anomaly model says "this part is unusual". Neither says "what happened, and does it matter".[1]
- **Fixed label lists miss real harms.** The standard benchmark has **no harassment class**, so a system built only on it can never report it.[1]
- **Adding more models can make it worse.** If you run five detectors and accept the first "yes", the false alarms of all five add up.
- **Research code is not a product.** No login, no queue, no storage, no report — and no way for a normal user to ask a follow-up question.

THE PROBLEM, IN ONE PARAGRAPH

> **Build a working platform that finds many kinds of real incidents in video — including ones with no fixed visual shape — reports them with correct timestamps and proof, keeps false alarms low even while using several models, and stays usable by a normal operator.**

WHICH SPLITS INTO FOUR QUESTIONS

- **Coverage** — one model cannot cover objects, actions and intent.
- **Trust** — with many models, who decides which answer wins?
- **Speed** — models take seconds to minutes; the UI cannot wait.
- **Proof** — an answer with no evidence is not useful.

Base paper

BASE PAPER

Real-World Anomaly Detection in Surveillance Videos

Sultani, W., Chen, C. & Shah, M. — CVPR 2018 · ref [1]

Statement. The authors say anomaly detection should be learned from *real* uncut CCTV footage, not staged clips. Labelling every frame is impossible, so they use **weak supervision**: a video is treated as a bag of segments, only the video-level label is known, and training pushes the most suspicious segment above the most normal one. They also released **UCF-Crime** — 1,900 real surveillance videos, about 128 hours, with 13 anomaly classes plus normal.

> **Why it is our base paper.** It defines the exact problem we are solving, and its dataset trained the action model that runs inside our system. So we do not only cite it — we run it.

WHAT WE TAKE FROM IT

- Incidents are **time spans**, not single frames.
- Its UCF-Crime label set, through our action model.
- Its own admitted weakness: many false alarms on normal video.

WHERE WE GO FURTHER

- Its 13 classes have **no harassment**, and the list cannot grow without new data and retraining.
- It produces a **score curve** — not a report, a clip, or an explanation.
- It uses one model, so nothing has to decide who is right.

Related papers

IMAGE 01

Figure from the base paper

optional — add a credit line

VideoMAE: Masked Autoencoders for Self-Supervised Video Pre-Training

Tong, Song, Wang & Wang — NeurIPS 2022 · ref [2]

Statement. Hide 90–95% of a video and train the model to rebuild it. The result is a strong video transformer that learns well even from small datasets, with no extra data needed.

> **We use it as:** the backbone of both our action models.

Scaling Open-Vocabulary Object Detection (OWLv2)

Minderer, Gritsenko & Houlsby — NeurIPS 2023 · ref [3]

Statement. Training on web image–text pairs lets a detector find objects described by *free text* at run time, instead of a fixed class list.

> **We use it as:** the zero-shot detector — search "a crowbar" with no retraining.

Gemini 1.5: Multimodal Understanding over Long Context

Gemini Team, Google — arXiv 2024 · ref [4]

Statement. A long-context multimodal model can watch a whole video directly and reason about it, instead of looking at a few sampled frames.

> **We use it as:** the main reasoning model — the only one that reads harassment from context.

optional: paste the paper's own diagram here.

Qwen2.5-VL Technical Report

Bai, S. et al. — arXiv 2025 · ref [5]

Statement. An open-weight vision-language model family with video understanding, time grounding and structured JSON output.

> **We use it as:** the offline backup — same output format, no API key, runs locally.

Learning Temporal Regularity in Video Sequences

Hasan, Choi, Neumann, Roy-Chowdhury & Davis — CVPR 2016 · ref [6]

Statement. Train an autoencoder on normal video only; anything it cannot rebuild well is treated as an anomaly. No anomalous training data needed.

> **Position:** the older approach our base paper beats — shown to explain why weak supervision replaced it.

Ultralytics YOLOv8

Jocher, Chaurasia & Qiu — 2023 · ref [7]

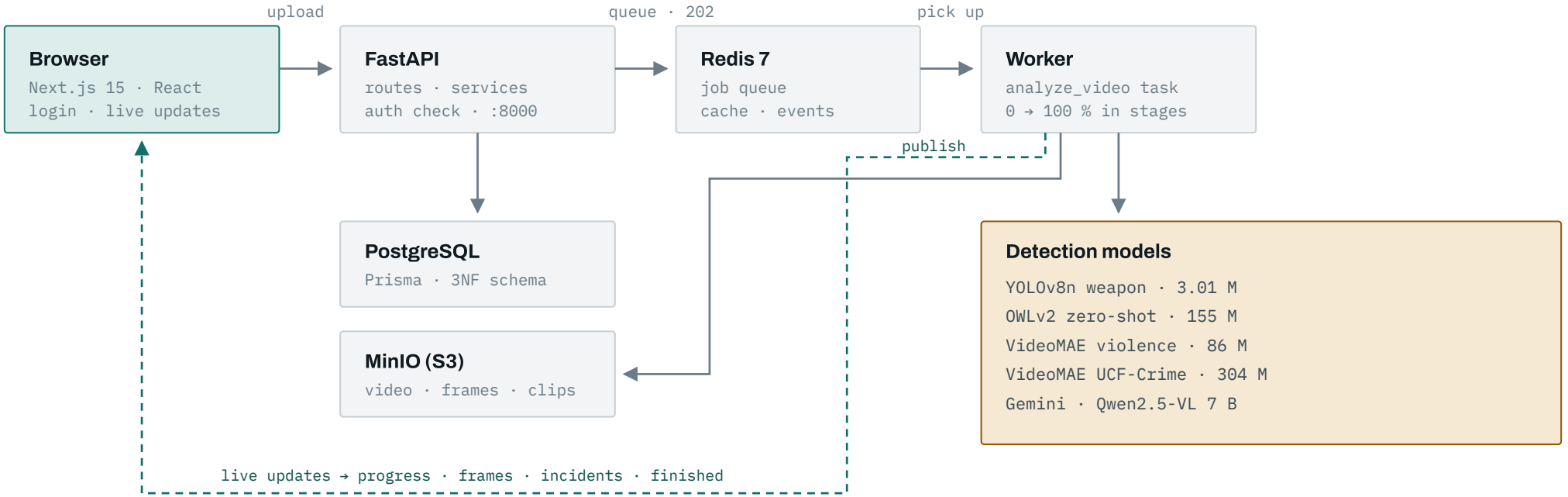
What the literature leaves open

SOURCE	GAP IT LEAVES	WHAT SENTINELAI DOES ABOUT IT
Sultani et al. [1] base paper	Fixed 13 classes, no harassment. The paper itself admits high false alarms. Gives scores, not incidents.	A language-vision model covers the missing types, and classifiers are moved to a backup role so their false alarms cannot overrule a good answer.
Tong et al. [2]	A backbone only — no report, no evidence, no user.	Wrapped in a sliding-window runner; output is merged into spans, clipped, boxed and saved.
Minderer et al. [3]	Frame-by-frame object hits, with no idea of an "event".	Hits are grouped and linked to stored frames and their timestamps.
Gemini [4] · Qwen-VL [5]	Strong but unmanaged: one is quota-limited and online, the other is slow. Neither says what to do when it is unavailable.	A clear fallback order, safe degradation when quota runs out, and a record of which model really answered.
General practice	Ensembles usually OR their "yes" answers together, so false alarms add up. Results are rarely searchable afterwards.	**Trust the best model first**, plus a clean database and a chat agent that can be asked about any past scan.

Our contribution, in one line. Not a new detector — a way of *deciding which detector to believe*, plus a merge step that removes duplicate and full-video alarms, and a searchable evidence store behind a live pipeline.

System architecture

The browser never talks to a model. It uploads, gets a job id in about one second, and then just **listens** while the worker sends progress and partial results.



CLEAN SPLIT
Pipeline code, model code and AI-agent code sit in three separate packages.

ONE DOOR
All six models are called through a single function, so the queue, the tests and the chat agent behave the same.

NEVER BLOCKS
Model work runs on a side thread, so progress keeps flowing while a model is busy.

Five stages, sent live as they finish

PROGRESS	STAGE	SENDS
0 – 5 %	Download the video, read its length	`job.started`
5 – 30 %	**Take preview frames** and save them	`frame.ready ×N`
30 – 70 %	Run the chosen detection model	`job.progress`
70 – 90 %	Cut incident clips, draw boxes	`segment.ready`
90 – 100 %	Add captions, save the final report	`job.completed`

Why frames come first. Reading a few frames takes under a second, but a model takes 5 seconds to several minutes. Doing frames first means the user sees real pictures at **1 second** instead of staring at a spinner.

HOW UPDATES REACH THE USER

- **One source of truth.** Every stage writes a row into a `job_events` table.
- **Two delivery paths, same data.** Redis sends each event to the browser (live stream) and to **signed webhooks**.
- **Safe to reconnect.** Each event has a number. If the connection drops, the browser sends the last number it saw and gets only the missing ones.
- **Fallback.** Clients that cannot hold a live connection can simply poll.

IMAGE 02

The `auto` chain — a fallback order, not a vote

Live progress card, mid-run
around 30–40 %

1 Gemini — the reasoning step

The only model that understands the scene instead of matching a pattern, and the only one that can read harassment. It sees the whole clip.

→ Its answer is FINAL — yes or no. The chain stops here.

2 Violence classifier

Runs only if step 1 could not answer at all — no API key, quota finished, or network down. Best on fights and assault.

→ A "yes" stops the chain; a "no" continues.

3 Action model (UCF-Crime)

Covers robbery, arson and road accidents. It tends to guess weak "Fighting" around 0.65 on normal clips, while real incidents score 0.95+.

→ So we raise its bar to 0.85.

4 Qwen2.5-VL — offline reasoning

Runs only if Gemini was unreachable *and* every classifier found nothing. Same output format, no API key, fully local — so the system still works with no internet.

→ Last resort, slow on CPU, 8 frames only.

Screen-record the analysis and take one frame from the middle.

THE KEY DESIGN DECISION

> **A trusted "no" is as valuable as a trusted "yes".** An earlier version took the first *yes* from any model. Gemini would watch a clip of a ball, correctly say "nothing here" — and that was thrown away the moment a weaker classifier guessed "violence".

Chaining models that way makes a *clear* video almost impossible to report, because every extra step only adds ways to say yes and none to say no. Ordering by capability keeps the false-alarm rate close to the *best* model instead of the worst.

SAFE DEGRADATION

Every step is optional. A missing key or a finished quota moves to the next step instead of failing the job — and the scan stores which model was *asked* and which one actually *answered*.

Two fixes between the model and the report

Raw model output is not a report yet. Every model passes through the same two steps.

STEP 1 — MERGE DUPLICATES

- One event described three times becomes three cards on screen. Overlapping and touching spans are **merged back into one incident**.
- Very low-confidence rows are dropped, and there is a limit on incidents per video.
- The final yes/no flag is recalculated after merging, so it can never disagree with the list.

STEP 2 — FOCUS ON THE PEAK

- An incident that covers the whole video marks every second as important — which is the same as marking none.
- Such spans are **pulled back to the most confident second**, and the description says so instead of shrinking silently.
- Object detectors skip both steps — they report frames, not spans.

RULES GIVEN TO THE LANGUAGE-VISION MODELS

- **Report each event once.** An argument that becomes a push and then a punch is one incident, not three.
- **Not incidents:** sport, martial-arts practice, children or animals playing, crowds, cheering.
- **Not weapons:** ball, bat, stick, tool, phone, bottle, toy. Only report a weapon you can actually see.
- **Honest confidence:** above 0.8 only when it is obvious, below 0.4 when the view is blocked.
- **A clear video is a normal result.** Reporting something that is not there is the worse mistake.

> All prompts are stored as editable `.md` files, not hard-coded text — change a rule, restart, and behaviour changes with no code edit.

Normalised to Third Normal Form

WHAT THE FIRST DESIGN GOT WRONG

PROBLEM	WHERE	BREAKS
A list inside one column	`labels TEXT[]`	1NF
Related data hidden in JSON	`result JSONB`	1NF
Storing a calculated count	`label_counts`	3NF
Free text used as a key	`model TEXT`	update anomaly

A label array cannot be indexed or fixed in one place. Data buried in JSON cannot be joined or counted. And a stored count is a second copy of a fact that will drift.[8]

THE NORMALISED DESIGN

```
users
 └ videos                (user_id)
    ├ analysis_jobs      (video_id, model_id)
    │   └ scans          (job_id, model_id)
    │      ├ segments    (scan_id, label_id, category_id)
    │      ├ detections  (scan_id, label_id, image_id)
    │      └ chat_sessions → messages → citations
    └ images             (video_id, segment_id)

detection_models · incident_categories · labels  ← lookups
job_events → webhook_deliveries → endpoints
```

- **No `user_id` on jobs or scans.** The owner is found through `job → video → user`. Storing it again would be the exact 3NF mistake we are avoiding.
- **Unique (user, file checksum)** on videos — uploading the same file twice becomes a lookup, not a second analysis.
- **Two model links per scan.** The job stores what was *asked for*; the scan stores what actually *answered*.

Three agents so the report can be questioned

An answer you cannot question is not useful. On top of the pipeline sits a chat layer that turns stored evidence into a conversation.

AGENT	WHAT IT DOES	WHAT KEEPS IT HONEST
`Orchestrator`	The chatbot the user talks to. Decides when to analyse a video, search the database, control playback or grab a frame.	Tool calls limited to 4 rounds, so it cannot loop forever.
`Database agent`	Turns a plain question into one **read-only SELECT** over that user's own scans, then explains the rows.	Limited schema view, row limit, and no write path exists at all.
`Scan chat`	Answers questions about *one* video — "what happened at 0:14?", "why was this flagged?"	Locked to that scan's summary, incidents and saved frames.

TOOLS THE CHATBOT CAN CALL

`ANALYZE_VIDEO` `QUERY_DETECTION_DATABASE` `CONTROL_VIDEO_PLAYBACK` `SHOW_FRAME_AT` `LIST_RECENT_SCANS` `GET_SCAN_REPORT`

Tools appear **only when they make sense** — video tools are hidden when no video is attached, so the model cannot call something it has no subject for.

IMAGE 03

Chat page — agent answering with a frame

/chat

Pick a turn where the agent used a tool and returned a picture.

Visual walkthrough

The screens in the order a real user meets them. Each dashed box is reserved for one screenshot.

To add an image, replace `<div class="shot-frame">…</div>` with `<img src="shots/01.png" alt="Login" style="width:100%;aspect-ratio:16/10;object-fit:cover">`

STEP 01
Sign up & OTP verify
/signup → /verify

Email OTP, hashed password, token issued.

STEP 02
Dashboard
/dashboard

Recent scans and incident counts.

STEP 03
Upload the video
/analyze

`auto` mode is the default.

STEP 04
Job accepted
job id in ~1 s

Returns before any model loads.

STEP 05
Live progress
frames appear at 30 %

Pictures arrive while the model still runs.

STEP 06
Incident timeline
/results

Type, confidence and peak second.

STEP 07
Evidence clip with boxes
annotated clip

The cut moment, boxes drawn on it.

STEP 08
Report
/report

The printable summary.

STEP 09
Ask about this scan
/chat

Answers come only from stored evidence.

STEP 10
Ask the database
"show scans with a weapon"

Agent writes one read-only query.

STEP 11
Admin panel
/admin

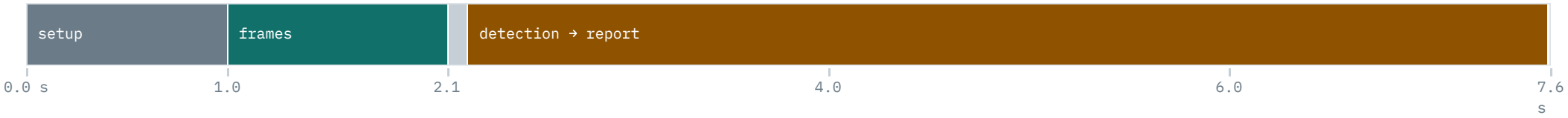
Users, scans and analytics.

STEP 12
Full architecture diagram
docs/architecture.mmd

The detailed version of slide 08.

The user sees content at 1 second, not 7.6

One real run on `weapon_gun.mp4` using the YOLO model, taken from the job's own event log.



0.0 s	`job.queued`	Accepted, id sent back
1.0 s	`frame.ready · 7 %`	**First picture on screen**
2.1 s	`frame.ready · 30 %`	All 12 frames shown
2.2 s	`job.progress · 40 %`	Model starts loading
7.6 s	`job.completed`	Report saved and delivered

FIRST CONTENT
1.0 s
instead of 7.6 s

WAITING TIME CUT
87 %
spinner removed

UPLOAD RESPONSE
~1 s
no model loaded yet

FRAMES STREAMED
12
before detection starts

Be clear about this. This measures *speed*, on one machine and one clip. It proves the async design works. It says nothing about detection accuracy — that is the next slide.

Why six models instead of one

MODEL	SIZE	WEAPONS	ANY OBJECT	ACTIONS	HARASSMENT	REASONING	OFFLINE	SPEED
YOLOv8n weapon	3.01 M	●	○	○	○	○	●	< 1 s
OWLv2	155 M	●	●	○	○	○	●	seconds
VideoMAE violence	86.2 M	○	○	●	○	○	●	seconds
VideoMAE action	303.9 M	○	○	●	○	○	●	seconds
Gemini	—	●	●	●	●	●	○	seconds
Qwen2.5-VL	7.6 B	●	●	◐	●	●	●	minutes (CPU)

THE COVERAGE POINT

No single row ticks every column. Classifiers are fast and offline but blind to intent. The two VLMs read intent but are either quota-limited or slow.

WHY QWEN IS LAST

It only sees **8 frames**, so a short incident between those frames is invisible to it. More frames = better recall, but linearly slower.

WHY THEY ARE SWAPPABLE

Gemini and Qwen-VL return the exact same JSON over the same five types, so one can replace the other with no change downstream.

Memory needed, by setup

Measured inside the container. Models load only when first used, then stay in memory.



Conclusion. Without the local 7B model the system runs fine in **8–16 GB**. With it, you need **32 GB** — so it is behind a build flag, off by default.

Detection accuracy — to be filled

Fill this from your own test videos before the defense. Real numbers, even small ones, are worth more than claimed numbers.

MODEL	PRECISION	RECALL	F1	FALSE ALARMS ON CLEAR CLIPS
YOLOv8n weapon	–	–	–	–
VideoMAE violence	–	–	–	–
VideoMAE action	–	–	–	–
Gemini	–	–	–	–
Qwen2.5-VL	–	–	–	–
auto chain	–	–	–	–

Suggested test set. About 30 clips that do contain an incident (spread over the five types) and about 30 hard clean clips — sport, play, crowds, objects that look like weapons. The strongest number to show is the **false-alarm rate on the clean clips**, because that is exactly what the design protects.

What the system does not do yet

MODELS AND ACCURACY

- **Depends on an online model.** The best step is an external API with a daily free quota. When it runs out, the system quietly drops to weaker models.
- **Limited frames offline.** The local VLM sees only 8 frames, so very short incidents can be missed.
- **Fixed label list in the fallback.** If the chain falls through to the action model, harassment coverage becomes zero.
- **No formal accuracy test yet** on a labelled set — the current claim is about design, not about numbers.
- **Confidence is what the model says**, not a calibrated probability.

SYSTEM

- **Uploads only, no live CCTV yet.** The design supports an RTSP camera feed, but that connection is not implemented — so today it is review after the fact, not live alerting.
- **CPU only.** The development machine has no usable GPU, so the local VLM takes minutes per clip.
- **Memory ceiling.** Turning on every local model needs about 22 GB RAM.
- **One worker pool** — no auto-scaling and no per-user limits yet.
- **No face recognition** — left out on purpose.

> **Ethics.** The system reports *events*, never *people*. No identification and no tracking across cameras — this is a design choice, not a missing feature.

Future work

NEXT STEP	WHAT IT GIVES
`Live CCTV (RTSP) input`	Turns the product from after-the-fact review into **real-time alerting** on a running camera.
`Proper accuracy test`	Turns the design claim into a measured one.
`Confidence calibration`	Makes scores mean the same thing across models.
`Train a harassment model`	Removes the dependence on an online model for the hardest type.
`GPU + smaller VLM`	Brings offline analysis from minutes to seconds.
`Operator feedback`	A confirm / dismiss button slowly builds a real labelled dataset.
`Multi-camera linking`	Follow one event across several camera views.

Conclusion

The useful step forward is no longer only a better detector. It is a **rule for deciding which detector to believe**, inside a system that can actually deliver and defend the answer.

- **Coverage** — six models covering objects, actions and intent, all behind one output format.
- **Trust** — a fixed order instead of a vote, so adding models no longer adds false alarms.
- **Speed** — staged processing with live updates: first picture at 1.0 s in a 7.6 s job.
- **Proof** — clean 3NF storage plus three chat agents, so any verdict can be questioned in plain language.

> **In one sentence.** A working platform that finds a wider range of real incidents than any single model in the literature — and knows which of its own answers to trust.

References

[1] Sultani, W., Chen, C. & Shah, M. **"Real-World Anomaly Detection in Surveillance Videos."** *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6479–6488. `Base paper`

[2] Tong, Z., Song, Y., Wang, J. & Wang, L. **"VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training."** *NeurIPS*, vol. 35, 2022.

[3] Minderer, M., Gritsenko, A. & Houlsby, N. **"Scaling Open-Vocabulary Object Detection."** *NeurIPS*, vol. 36, 2023.

[4] Gemini Team, Google. **"Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context."** *arXiv preprint* arXiv:2403.05530, 2024. arxiv.org/abs/2403.05530

[5] Bai, S. *et al.* **"Qwen2.5-VL Technical Report."** *arXiv preprint* arXiv:2502.13923, 2025. arxiv.org/abs/2502.13923

[6] Hasan, M., Choi, J., Neumann, J., Roy-Chowdhury, A. K. & Davis, L. S. **"Learning Temporal Regularity in Video Sequences."** *CVPR*, 2016, pp. 733–742.

[7] Jocher, G., Chaurasia, A. & Qiu, J. **"Ultralytics YOLOv8."** Software release, 2023. github.com/ultralytics/ultralytics

[8] Codd, E. F. **"Further Normalization of the Data Base Relational Model."** *Data Base Systems, Courant Computer Science Symposia Series 6*, Prentice-Hall, 1972, pp. 33–64.

Before you submit: reformat these into your department's style (IEEE / APA) and open every link once to check it works.